

Lab # 13b

Shell Scripting Using Functions

Function Statement

```
Syntax
function_name(){
    # Algo..
}
```

OR

```
function function_name {
    # Algo..
}
```

Function Calling

```
function_name
```

I. Shell functions can be used to group commands in a logical way.

```
$ cat funcs.sh
#!/bin/bash
```

```
function greetings {
    echo Hello World!
    echo and hello to $USER to!
}
```

We will now call a function greetings echo The end

This is sample output from this script with a function.

```
./funcs.sh
```

II. A shell function can also receive parameters.

```
$ cat addfunc.sh
```

```
#!/bin/bash
```

```
function plus {  
  let result="$1 + $2"  
  echo $1 + $2 = $result  
}
```

Calling

```
plus 3 10  
plus 20 13  
plus 20 22
```

Example

```
#!/bin/bash
```

```
# Function to add two numbers
```

```
add() {  
  local num1=$1  
  local num2=$2  
  echo $((num1 + num2))  
}
```

```
# Function to subtract two numbers
```

```
subtract() {  
  local num1=$1  
  local num2=$2  
  echo $((num1 - num2))  
}
```

```
# Function to multiply two numbers
```

```
multiply() {  
  local num1=$1  
  local num2=$2  
  echo $((num1 * num2))  
}
```

```
# Function to divide two numbers
divide() {
    local num1=$1
    local num2=$2
    if [ $num2 -ne 0 ]; then
        echo $((num1 / num2))
    else
        echo "Division by zero error"
    fi
}

# Main script logic
while true;
do
    echo "Select an operation:"
    echo "1. Add"
    echo "2. Subtract"
    echo "3. Multiply"
    echo "4. Divide"
    echo "5. Exit"
    read -p "Enter your choice: " choice

    if [ "$choice" -eq 5 ]; then
        echo "Exiting..."
        break
    fi

    read -p "Enter first number: " num1
    read -p "Enter second number: " num2
```

```
case $choice in
```

```
1)
```

```
    result=$(add $num1 $num2)
```

```
    echo "Result: $num1 + $num2 = $result"
```

```
    ;;
```

```
2)
```

```
    result=$(subtract $num1 $num2)
```

```
    echo "Result: $num1 - $num2 = $result"
```

```
    ;;
```

```
3)
```

```
    result=$(multiply $num1 $num2)
```

```
    echo "Result: $num1 * $num2 = $result"
```

```
    ;;
```

```
4)
```

```
    result=$(divide $num1 $num2)
```

```
    echo "Result: $num1 / $num2 = $result"
```

```
    ;;
```

```
*)
```

```
    echo "Invalid choice, please try again."
```

```
    ;;
```

```
esac
```

```
done
```

To find Odd or Even no.

```
a=2
b=0
echo Enter the no.
read x
b=`expr $x % $a`
if [ $b -eq 0 ]
then
echo Given no. is Even
else
echo Given no. is Odd
```

To reverse a no.

```
echo Enter the no.
read x
while [ $x -gt 0 ]
do
c=`expr $x % 10`
echo $c
x=`expr $x/10`
done
```

To print the fibonnicii series

```
count=1
f1=1
f2=1
k=0
echo $f1
echo $f2
while [ $count -le 10 ]
do
k=`expr $f1 + $f2`
echo $k
f1=$f2
f2=$k
count=`expr $count + 1`
done
```

To find average of two nos.

```
echo enter the 1st no.
read a
echo enter the 2nd no.
read b
c=`expr $a + $b`
d=$c/2
echo the average of two no. is $d
```

1. Implement all the above logic using functions